NetEvidence AI

Evidence-Based IP Network-Type Classification

Project detail	Information
Hackathon	AIORI Hackathon 3
Problem Statement	PS-010 — Evidence-Based IP Network-Type Classification
Team	Evolve
Team Code	A3-PS010-TC192
Institution	Guru Nanak Institutions Technical Campus (GNITC)
Report scope	Project status, observed demo features, design, and remaining work

Classify with evidence. Explain uncertainty. Make every result auditable.

1. Executive Summary

NetEvidence AI is a web application intended to classify the likely network type associated with an IP address by presenting network evidence and explaining uncertainty. The project addresses a technical challenge: an IP address or autonomous system does not, by itself, always establish whether a connection is residential, mobile, enterprise/campus, or hosting/datacenter.

The application demonstration visibly includes an IP-analysis interface, malformed-IP validation, predefined evaluator scenarios, an Evidence Explorer with a topology graph, an analysis-history page, a benchmarking page, and an Audit Trail interface. These are observations from the screen recording, not proof that every underlying function is connected to live data.

A key qualification is that the recording shows an evaluator/demo mode using synthetic scenarios. The video therefore supports the claim that these interfaces and demo flows are present, but it does not establish the accuracy of real-world IP classification, dynamic multi-source evidence fusion, or live conflict detection.

The current priority is to verify backend behaviour and actual data sources, complete frontend/backend integration, resolve the GitHub repository-access issue for Vercel, and evaluate the classification engine using labelled test data.

2. Problem Statement

The project aims to determine the likely network type associated with an IP address when network information may be incomplete, ambiguous, outdated, or contradictory.

- Ambiguous network ownership: an operator or ASN may serve multiple kinds of customers.
- Incomplete information: some addresses may lack useful routing, reverse-DNS, or IP-intelligence data.
- Conflicting clues: different sources may suggest different network-use categories.
- Limited explainability: a category label without supporting evidence can be difficult to assess.
- Uncertainty: a reliable system should be able to return Unknown when evidence is insufficient.

Core challenge: assess what the available evidence supports and communicate limitations instead of overstating certainty.

3. Proposed Solution

NetEvidence AI is designed around an evidence-centred analysis workflow. It accepts an IP address, validates it, obtains available network information from configured sources, organises evidence, evaluates consistency, and presents a likely network type with supporting information.

3.1 Intended components

- IP input and validation: accepts user input and visibly rejects malformed input in the demo.
- Evidence collection: intended to use network information such as ASN, BGP/prefix details, reverse DNS, or approved IP-intelligence sources; live collection must be verified.
- Classification: the interface represents Residential, Hosting/Datacenter, Enterprise/Campus, Mobile, and Unknown scenarios.
- Evidence Explorer: the video shows a topology graph with an IP, BGP routing, a routed subnet, an autonomous system, and reverse-DNS-related nodes.
- Conflict handling: a “Host vs ISP — Conflicting Signals” test case and a “Conflicting Signals” scenario are visible; dynamic detection from real data remains unverified.

- Insufficient-evidence handling: an “Unknown / Insufficient Evidence” scenario is visible; whether this result is produced dynamically remains unverified.
- History and audit: dedicated history and Audit Trail interfaces are shown; persistence and data integrity require testing.
- Benchmarking: a benchmarking page is shown; its dataset, methodology, and validity require verification.

4. What the Screen Recording Demonstrates

Observed item	What is visible	What it establishes
Dashboard	Overview cards, network-type visualisation, recent-analysis area	Dashboard interface exists; calculations need verification
IP analysis	IP input, Analyze IP action, validation/error display, example cases	Analysis UI and input-validation flow are present
Malformed IP	Input 76.465.732 is rejected as malformed	Visible evidence of input validation
Evidence Explorer	Predefined network-type and conflicting/insufficient-evidence scenarios	Scenario-driven interface is present
Topology graph	IP, BGP routing, subnet, ASN, reverse-DNS-related nodes	Graph rendering is demonstrated; live provenance is not
History	History page in navigation	Page exists; working persistence is unverified
Benchmarking	Benchmarking page in navigation	Page exists; methodology/results need verification
Audit Trail	Events, IP addresses, statuses, metadata visible	Audit interface and displayed records are demonstrated

5. What Makes NetEvidence AI Different

The strongest potential differentiator is not simply displaying an IP address, ISP, or ASN. It is the intended combination of evidence relationships, uncertainty handling, conflict analysis, and an audit trail in one workflow.

- Evidence topology: the graph visually links an IP with routing, subnet, ASN, and reverse-DNS-related information.
- Conflict-aware workflow: the interface includes a host-versus-ISP conflicting-signals test scenario.
- Unknown outcome: the interface includes an insufficient-evidence scenario rather than only displaying a forced category.
- Audit trail: the interface displays analysis events and metadata.
- Unified explanation: the intended product brings evidence and classification into one interface.

6. Existing Approaches and Comparison

Existing IP-intelligence services and routing-data tools serve different purposes. Some provide IP geolocation or network/hosting attributes; routing tools provide information about prefixes and autonomous systems; reverse DNS can offer naming clues. These signals do not always establish the exact end-user connection type.

Aspect	Existing tools (capabilities vary)	NetEvidence AI approach/status
IP/network metadata	Provided by many IP-intelligence and routing tools	Intended as inputs to analysis; live sources need verification
Evidence relationships	Presentation depends on product	Topology graph visibly rendered in demo
Conflict handling	Capabilities vary	Conflicting scenario shown; dynamic operation unverified
Insufficient evidence	Behaviour varies	Unknown scenario shown; dynamic operation unverified
Audit/history	Product-dependent	History and Audit Trail interfaces shown; persistence unverified
Accuracy	Must be assessed per product and use case	No measured accuracy established by this video

7. Technology and Architecture

- Frontend: React, Vite, TypeScript, and Tailwind CSS.
- Backend: Python and FastAPI.
- Local data: SQLite is used in the local development setup described for the project.
- Source control: Git and GitHub.
- Intended deployment: Vercel for the frontend, with backend hosting/configuration to be confirmed.

8. Work Completed So Far

- A frontend application with dashboard, IP analysis, Evidence Explorer, history, benchmarking, Audit Trail, settings/help navigation is present in the demo.
- IP input validation is visibly demonstrated by rejection of a malformed IP address.
- Predefined evaluator scenarios include network-type examples, conflicting signals, and insufficient evidence.
- The Evidence Explorer graph is visibly rendered with network-related nodes and links.
- Audit Trail records are visible in the demonstrated interface.
- The project has a frontend/backend codebase and local development setup described in the existing project context.

9. Remaining Work

9.1 Live IP analysis and backend verification

- Test valid public IPv4 and IPv6 addresses with demo mode disabled.
- Confirm which network data sources are actually queried and whether their responses are live.
- Check that the classifier consumes those responses rather than returning predefined scenario data.
- Verify how confidence and evidence-quality values are calculated.

9.2 Evidence fusion and conflict detection

- Inspect whether evidence from multiple sources is normalized and combined.
- Test whether conflicts are detected dynamically from actual source responses.
- Verify that Unknown results are triggered by defined sufficiency rules.
- Confirm that topology edges correspond to collected evidence and preserve source provenance.

9.3 Integration and persistence

- Verify frontend requests, API responses, error handling, and external-source failures.
- Test whether analysis history and audit events persist after refresh/restart as intended.
- Confirm the database strategy is suitable for production hosting.

9.4 Deployment

- Grant the Vercel GitHub App access to the correct organisation repository or use an authorised repository route.
- Build the frontend for production and configure the deployed backend API URL.
- Configure CORS, environment variables, backend hosting, and persistent storage.
- Verify the complete public application after deployment.

9.5 Evaluation

- Create a labelled test set with documented labelling rules.
- Measure accuracy, precision, recall, F1-score, and per-class confusion.
- Test missing data, conflicting evidence, and unavailable-source scenarios.
- Compare against selected existing tools using the same examples.
- Avoid accuracy or superiority claims until measurements are available.

10. Risks and Limitations

- Demo-data risk: synthetic scenarios may make an interface look functional without proving live evidence collection.
- Network-type ambiguity: an ASN or hosting flag may not determine the exact customer connection type.
- Source availability: coverage, freshness, rate limits, API credentials, and licensing may affect results.
- Confidence interpretation: a displayed confidence value is not necessarily a calibrated probability.
- Deployment configuration: local URLs, environment variables, CORS, and database persistence need production checks.

11. Testing Plan

- Functional: valid/invalid IP inputs, API responses, frontend/backend communication, and error handling.
- Evidence: missing, stale, contradictory, and unavailable-source responses.
- Classification: labelled examples across all supported categories.
- Audit/history: verify event creation, persistence, and correct association with each analysis.
- Deployment: test the public URL from a clean browser session and verify API connectivity.

12. Future Enhancements

- Expand permitted network-evidence sources and document provenance.

- Improve evidence-quality and confidence validation.
- Add a clear explanation of each conflict and its effect on the result.
- Show timestamps and freshness for individual evidence items.
- Implement historical evidence snapshots and comparisons.
- Add reproducible benchmarks and publish measured limitations.

13. Expected Impact

If implemented and validated, NetEvidence AI could help network analysts and developers inspect IP-related network context with more transparent evidence and explicit uncertainty. Potential uses include network analytics, internet measurement, infrastructure analysis, and security-investigation support. The output should support analysis, not be treated as definitive proof of a person's identity, location, or behaviour.

14. Conclusion

The screen recording confirms that NetEvidence AI has a developed interface with IP input validation, evaluator scenarios, an evidence topology visualisation, and history/benchmark/audit sections. The most promising product direction is to make network-type classification explainable by showing relationships among evidence and handling conflicts and insufficient information.

The next critical step is to verify that these workflows are driven by live, correctly attributed evidence rather than synthetic demonstration data. The team should then complete integration and Vercel deployment, test persistent history and audit records, and evaluate the classifier against labelled data. Until that work is done, the project should present evidence visualisation and scenario-driven workflows as demonstrated, while describing live evidence fusion, dynamic conflict detection, and classification performance as unverified.

NetEvidence AI — Evidence-based. Explainable. Uncertainty-aware.